Apollo Checklist

Closure XXXVIII — DCT-I and DST-I Forward Known-Value Fixtures [patch]

Sprint target version: 0.12.18

- ☑ Triage: identify remaining DCT-I/DST-I published-reference gaps (forward known-value fixtures).
- ☑ Add validation fixture 58: `dct1_three_point_forward_known_values_fixture` (DCT-I N=3 x=[1,2,3]: y=[8,−2,0]; boundary formula; Rao & Yip 1990 Table 2.1; threshold 1e-15).
- ☑ Add validation fixture 59: `dst1_two_point_forward_known_values_fixture` (DST-I N=2 x=[1,3]: y=[4√3,−2√3]; formula y[k]=2·Σsin; Rao & Yip 1990 Table 3.1; threshold 1e-12).
- ☑ Update `run_published_reference_suite` call list: 57 → 59 fixtures.
- ☑ Update both count assertions in `apollo-validation/suite.rs`: 57 → 59.
- ☑ Update root `README.md` fixture count 57 → 59; append two new entries.
- ☑ Update sprint PM artifacts: CHANGELOG.md, backlog.md, checklist.md, gap_audit.md.
- ☑ `cargo test -p apollo-validation`: 3 passed, 0 FAILED, 0 ignored.

Closure XXXVII — DCT-III and DST-III Published-Reference Fixtures [patch]

Sprint target version: 0.12.17

- ☑ Triage: identify remaining DCT-III/DST-III published-reference gaps.
- ☑ Add validation fixture 56: `dct3_dc_input_flat_output_fixture` (DCT-III N=4 DC input [1,0,0,0]: y=[½,½,½,½]; Makhoul 1980 Table I; FFTW REDFT01; threshold 1e-15).
- ☑ Add validation fixture 57: `dst3_nyquist_input_alternating_output_fixture` (DST-III N=4 Nyquist input [0,0,0,1]: y=[½,−½,½,−½]; Makhoul 1980 Table II; FFTW RODFT01; threshold 1e-15).
- ☑ Update `run_published_reference_suite` call list: 55 → 57 fixtures.
- ☑ Update both count assertions in `apollo-validation/suite.rs`: 55 → 57.
- ☑ Update root `README.md` fixture count 55 → 57; append two new entries.
- ☑ Update sprint PM artifacts: CHANGELOG.md, backlog.md, checklist.md, gap_audit.md.
- ☑ `cargo test -p apollo-validation`: 3 passed, 0 FAILED, 0 ignored.

Closure XXXVI — CWT Ricker Impulse Peak and Scale-Normalization Fixtures [patch]

Sprint target version: 0.12.16

- ☑ Triage: identify remaining CWT published-reference gaps (impulse response value, L^2 scale normalization).
- ☑ Add validation fixture 54: `cwt_ricker_impulse_peak_value_fixture` (N=7 a=1 δ_{3}: W(1,3)=ψ(0), W(1,2)=W(1,4)=0; Daubechies 1992 §2.1 eq.(2.1.4); threshold 1e-14).

- ☑ Add validation fixture 55: `cwt_ricker_scale_normalization_fixture` (N=7 a=2 δ_{3}: W(2,3)=ψ(0)/√2; Daubechies 1992 §2.1 / Grossmann-Morlet 1984 eq.(1.3); threshold 1e-13).
- ☑ Update imports: add `ContinuousWavelet, CwtPlan` to `apollo-validation` use statement.
- ☑ Update `run_published_reference_suite` call list: 53 → 55 fixtures.
- ☑ Update both count assertions in `apollo-validation/suite.rs`: 53 → 55.
- ☑ Update root `README.md` fixture count 53 → 55; append two new entries.
- ☑ Update sprint PM artifacts: CHANGELOG.md, backlog.md, checklist.md, gap_audit.md.
- ☑ `cargo test -p apollo-validation`: 3 passed, 0 FAILED, 0 ignored.

Closure XXXV — Daubechies-4 DWT Coefficient and Reconstruction Fixtures [patch]

Sprint target version: 0.12.15

- ☑ Triage: identify remaining wavelet published-reference gaps (db4 coefficients and db4 PR roundtrip).
- ☑ Add validation fixture 52: `wavelet_daubechies4_one_level_known_coefficients_fixture` (db4 N=4 level=1 x=[1,0,0,0]: [a0,a1,d0,d1]=[h0,h2,h3,h1]; Daubechies 1992 taps; threshold 1e-15).
- ☑ Add validation fixture 53: `wavelet_daubechies4_inverse_perfect_reconstruction_fixture` (db4 N=4 level=1: IDWT(DWT([1,-2,0.5,4]))=[1,-2,0.5,4]; Mallat 1989 Thm.2; threshold 1e-12).
- ☑ Update `run_published_reference_suite` call list: 51 → 53 fixtures.
- ☑ Update both count assertions in `apollo-validation/suite.rs`: 51 → 53.
- ☑ Update root `README.md` fixture count 51 → 53; append two new entries.
- ☑ Update sprint PM artifacts: CHANGELOG.md, backlog.md, checklist.md, gap_audit.md.
- ☑ `cargo test -p apollo-validation`: 3 passed, 0 FAILED, 0 ignored.

Closure XXXIV — CZT Off-Unit-Circle and Hilbert Envelope Fixtures [patch]

Sprint target version: 0.12.14

- ☑ Triage: identify remaining published-reference gaps (CZT A≠1, Hilbert envelope).
- ☑ Add validation fixture 50: `czt_off_unit_circle_z_transform_fixture` (N=2 A=2 W=exp(-πi); Z{x}(2)=1.5, Z{x}(-2)=0.5; exact dyadic; threshold 1e-12).
- ☑ Add validation fixture 51: `hilbert_pure_cosine_envelope_is_unity_fixture` (cos(πn/2) N=4; envelope=[1,1,1,1]; exact integers; threshold 1e-12).
- ☑ Update `run_published_reference_suite` call list: 49 → 51 fixtures.
- ☑ Update both count assertions in `apollo-validation/suite.rs`: 49 → 51.
- ☑ Update root `README.md` fixture count 49 → 51; append two new entries.
- ☑ Update sprint PM artifacts: CHANGELOG.md, backlog.md, checklist.md, gap_audit.md.
- ☑ `cargo test -p apollo-validation`: 3 passed, 0 FAILED, 0 ignored.

Closure XXXIII — SDFT Sliding Recurrence and FrFT Order-4 Identity Fixtures [patch]

Sprint target version: 0.12.13

- ☑ Triage: identify next published-reference gaps (SDFT sliding path, FrFT periodicity).
- ☑ Add validation fixture 48: `sdft_sliding_recurrence_unit_impulse_all_bins_fixture` (N=4 zero_state, feed [1,0,0,0], all bins=1+0i; Jacobsen-Lyons 2003 eq.(2); exact; threshold 1e-12).
- ☑ Add validation fixture 49: `frft_order4_identity_fixture` (UnitaryFrFT N=4 order=4.0: output=[1,2,3,4]=input; Candan 2000 §II Corollary; exact; threshold 1e-12).
- ☑ Update `run_published_reference_suite` call list: 47 → 49 fixtures.
- ☑ Update both count assertions in `apollo-validation/suite.rs`: 47 → 49.
- ☑ Update root `README.md` fixture count 47 → 49; append two new entries.
- ☑ Update sprint PM artifacts: CHANGELOG.md, backlog.md, checklist.md, gap_audit.md.
- ☑ `cargo test -p apollo-validation`: 3 passed, 0 FAILED, 0 ignored.

Closure XXXII — NUFFT Adjoint Identity and Radon Fourier Slice Theorem Fixtures [patch]

Sprint target version: 0.12.12

- ☑ Triage: identify remaining published-reference gaps (NUFFT adjoint, Radon FST).
- ☑ Add validation fixture 46: `nufft_type1_type2_adjoint_inner_product_fixture` (N=2 adjoint Re(⟨Ac,f⟩)=Re(⟨c,A*f⟩)=5; Dutt-Rokhlin 1993; exact; threshold 1e-12).
- ☑ Add validation fixture 47: `radon_fourier_slice_theorem_theta0_fixture` (Radon θ=0 FST on [[1,2],[3,4]]; Natterer 1986 Thm 1.1; exact; threshold 1e-12).
- ☑ Update `run_published_reference_suite` call list: 45 → 47 fixtures.
- ☑ Update both count assertions in `apollo-validation/suite.rs`: 45 → 47.
- ☑ Update root `README.md` fixture count 45 → 47; append two new entries.
- ☑ Update sprint PM artifacts: CHANGELOG.md, backlog.md, checklist.md, gap_audit.md.
- ☑ `cargo test -p apollo-validation`: 3 passed, 0 FAILED, 0 ignored.

Closure XXXI — DCT-I and DST-I Self-Inverse Published-Reference Fixtures [patch]

Sprint target version: 0.12.11

- ☑ Triage: identify DCT/DST family remaining gaps (DCT-I, DST-I inverse-roundtrip fixtures absent).
- ☑ Add validation fixture 44: `dct1_inverse_roundtrip_three_point_fixture` (DCT-I N=3, IDCT-I∘DCT-I=I, Makhoul 1980 $C1^2=2(N-1)\cdot I$, FFTW REDFT00, threshold 1e-14).
- ☑ Add validation fixture 45: `dst1_inverse_roundtrip_two_point_fixture` (DST-I N=2, IDST-I∘DST-I=I, Makhoul 1980 $S1^2=2(N+1)\cdot I$, FFTW RODFT00, threshold 1e-14).
- ☑ Update `run_published_reference_suite` call list: 43 → 45 fixtures.
- ☑ Update both count assertions in `apollo-validation/suite.rs`: 43 → 45.
- ☑ Update root `README.md` fixture count 43 → 45; append two new entries.
- ☑ Update sprint PM artifacts: CHANGELOG.md, backlog.md, checklist.md, gap_audit.md.
- ☑ `cargo test -p apollo-validation -p apollo-dctdst`: 0 FAILED, 0 ignored.

Closure XXX — DCT-IV and DST-IV Self-Inverse Published-Reference Fixtures [patch]

Sprint target version: 0.12.10

- ☑ Triage: identify DCT/DST family gaps (DCT-IV, DST-IV inverse-roundtrip fixtures absent).
- ☑ Add validation fixture 42: `dct4_inverse_roundtrip_two_point_fixture` (DCT-IV N=2, IDCT-IV∘DCT-IV=I, Makhoul 1980 $C4^2$=N·I, FFTW REDFT11, threshold 1e-14).
- ☑ Add validation fixture 43: `dst4_inverse_roundtrip_two_point_fixture` (DST-IV N=2, IDST-IV∘DST-IV=I, Makhoul 1980 $S4^2$=N·I, FFTW RODFT11, threshold 1e-14).
- ☑ Update `run_published_reference_suite` call list: 41 → 43 fixtures.
- ☑ Update both count assertions in `apollo-validation/suite.rs`: 41 → 43.
- ☑ Update root `README.md` fixture count 41 → 43; append two new entries.
- ☑ Update sprint PM artifacts: CHANGELOG.md, backlog.md, checklist.md, gap_audit.md.
- ☑ `cargo test --workspace`: 0 FAILED, 0 ignored.

Closure XXIX — Inverse-Roundtrip Published-Reference Fixtures: NTT, STFT [patch]

Sprint target version: 0.12.9

- ☑ Triage: identify transforms with inverse API but no inverse-roundtrip fixture (NTT, STFT).
- ☑ Add validation fixture 40: `ntt_inverse_roundtrip_fixture` (NTT N=4, INTT∘NTT=I in Z/pZ, Pollard 1971, threshold 1e-12).
- ☑ Add validation fixture 41: `stft_hann_wola_inverse_roundtrip_fixture` (STFT frame=4 hop=2, Hann COLA WOLA, Allen-Rabiner 1977, threshold 1e-12).
- ☑ Update `run_published_reference_suite` call list: 39 → 41 fixtures.
- ☑ Update both count assertions in `apollo-validation/suite.rs`: 39 → 41.
- ☑ Update root `README.md` fixture count 39 → 41; append two new entries.
- ☑ Update sprint PM artifacts: CHANGELOG.md, backlog.md, checklist.md, gap_audit.md.
- ☑ `cargo test --workspace`: 0 FAILED, 0 ignored.

Closure XXVIII — Inverse-Roundtrip Published-Reference Fixtures: DHT, SFT [patch]

Sprint target version: 0.12.8

- ☑ Triage: identify transforms with inverse API but no inverse-roundtrip fixture (DHT, SFT).
- ☑ Add validation fixture 38: `dht_inverse_roundtrip_fixture` (DHT N=4, IDHT∘DHT=I, Bracewell 1983, threshold 1e-14).
- ☑ Add validation fixture 39: `sft_inverse_roundtrip_fixture` (SFT N=4 K=1, ISFT∘SFT=I, Hassanieh et al. 2012, threshold 1e-12).
- ☑ Update `run_published_reference_suite` call list: 37 → 39 fixtures.
- ☑ Update both count assertions in `apollo-validation/suite.rs`: 37 → 39.
- ☑ Update root `README.md` fixture count 37 → 39; append two new entries.
- ☑ Update sprint PM artifacts: CHANGELOG.md, backlog.md, checklist.md, gap_audit.md.
- ☑ `cargo test --workspace`: 0 FAILED, 0 ignored.

Closure XXVII — Inverse-Roundtrip Published-Reference Fixtures: FWHT, QFT, SHT [patch]

Sprint target version: 0.12.7

- ☑ Triage: identify transforms with inverse API but no inverse-roundtrip fixture.
- ☑ Add validation fixture 35: `fwht_inverse_roundtrip_fixture` (FWHT N=4, IFWHT∘FWHT=I, Walsh 1923, threshold 1e-14).
- ☑ Add validation fixture 36: `qft_inverse_roundtrip_fixture` (QFT N=4, iqft∘qft=I, Shor 1994, threshold 1e-12).
- ☑ Add validation fixture 37: `sht_inverse_roundtrip_y10_fixture` (SHT lmax=1, dipole Y_1^0 roundtrip, Driscoll-Healy 1994, threshold 1e-10).
- ☑ Update `run_published_reference_suite` call list: 34 → 37 fixtures.
- ☑ Update both count assertions in `apollo-validation/suite.rs`: 34 → 37.
- ☑ Update root `README.md` fixture count 34 → 37; append three new entries.
- ☑ Update sprint PM artifacts: CHANGELOG.md, backlog.md, checklist.md, gap_audit.md.
- ☑ `cargo test --workspace`: 0 FAILED, 0 ignored.

Closure XXVI — Inverse-Roundtrip Published-Reference Fixtures: DWT, GFT, FrFT [patch]

Closure XXIV — GPU Adapter Preference, Test Runtime-Skip, Bluestein CZT Fix [patch]

Closure XXV — Hilbert Instantaneous Frequency + Doc/Test/PM Cleanup [patch]

- ☑ Convert `apollo-ntt-wgpu` doc-test from `rust,ignore` to `rust,no_run` with preamble; verify 0 ignored workspace-wide.
- ☑ Expand `execute_inverse_with_buffers` doc comment in `apollo-stft-wgpu/device.rs`.
- ☑ Add missing `CHANGELOG.md` entries for Closure XXIII (0.12.3) and Closure XXIV (0.12.4).
- ☑ Implement `AnalyticSignal::instantaneous_frequency()` using complex-derivative formula.
- ☑ Add `instantaneous_frequency_constant_tone` test (ε<1e-10, k/N=5/64).
- ☑ Add `double_hilbert_negates_zero_mean_signal` test (ε<1e-10, sinusoidal input N=32).
- ☑ Add validation fixture 31: `hilbert_instantaneous_frequency_constant_tone_fixture` (N=64, k=5, threshold 1e-10).
- ☑ Update `run_published_reference_suite` call list to include fixture 31.
- ☑ Update `assert_eq!(report.external.published_references.attempted, 30)` → 31.
- ☑ Update `assert_eq!(report.attempted, 30)` → 31.
- ☑ Update root `README.md` fixture count: 30 → 31; append new fixture entry.
- ☑ Update `apollo-hilbert/README.md` with instantaneous frequency subsection.
- ☑ `cargo test -p apollo-hilbert`: 11 passed, 0 failed, 0 ignored.
- ☑ `cargo test -p apollo-validation`: 3 passed, 0 failed, 0 ignored.
- ☑ `cargo test --workspace`: 0 FAILED, 0 ignored.
- ☑ Artifact sync: backlog.md, checklist.md, gap_audit.md, CHANGELOG.md.

Closure XXIV — GPU Adapter Preference, Test Runtime-Skip, Bluestein CZT Fix [patch]

Sprint target version: 0.12.4

- ☑ Replace all 20 `wgpu::RequestAdapterOptions::default()` with `PowerPreference::HighPerformance`.
- ☑ Remove `#[ignore]` from all 10 `apollo-ntt-wgpu` tests; convert to early-return pattern.
- ☑ Remove `#[ignore]` from all 7 `apollo-stft-wgpu` tests (early-return pattern already present).
- ☑ Fix `stft_chirp.wgsl`: premul_fwd sign ($-\pi i \cdot n^2/N$), premul_inv sign ($+\pi i \cdot n^2/N$), postmul_fwd sign ($-\pi i \cdot k^2/N$), postmul_inv sign ($+\pi i \cdot n^2/N$).
- ☑ Add `stft_chirp_pointmul_fwd` entry point (conj(h_stored) = h_fwd via −h_fft_im).
- ☑ Add `pointmul_fwd_pipeline` field to `StftChirpData`; build in `new()`.
- ☑ Update `execute_forward_fft_chirp` in `kernel.rs` to dispatch `pointmul_fwd_pipeline`.
- ☑ Add POT guard to `execute_forward_with_buffers` delegating non-PoT to allocating path.
- ☑ Add POT guard to `execute_inverse_with_buffers` delegating non-PoT to allocating path.
- ☑ Calibrate forward CZT test tolerance: 1e-2 → 2e-2 (f32 GPU argument-reduction, N=400).
- ☑ `cargo check --workspace`: 0 errors, 0 warnings.
- ☑ `cargo test -p apollo-stft-wgpu`: 23/23 passed, 0 failed, 0 ignored.
- ☑ `cargo test --workspace`: 0 FAILED (case-sensitive), 0 ignored.
- ☑ Artifact sync: backlog.md, checklist.md, gap_audit.md.

Closure XXIII — ARCHITECTURE.md Capability Annotation + Validation Fixtures 29-30 [patch]

Sprint target version: 0.12.3

- ☑ Audit ARCHITECTURE.md Mixed-Precision Capability Table for stale Notes entries.
- ☑ Fix `apollo-czt-wgpu` Notes: "forward + inverse CZT; f16 promoted to f32 at host boundary".
- ☑ Fix `apollo-mellin-wgpu` Notes: "forward + inverse Mellin spectrum; f16 promoted at host boundary".
- ☑ Add `czt_inverse_vandermonde_roundtrip_fixture` (fixture 29): N=4, A=1, W=exp(-2πi/4), threshold 1e-12; Björck-Pereyra (1970) + Rabiner-Schafer-Rader (1969).
- ☑ Add `mellin_inverse_spectrum_constant_roundtrip_fixture` (fixture 30): N=32, c=2, [1,4], threshold 1e-10; Mellin (1896), Titchmarsh (1937).
- ☑ Add `published_real_fixture_with_threshold` helper; refactor `published_real_fixture` to delegate.
- ☑ Register both fixtures in `run_published_reference_suite`.
- ☑ Update `validation_suite_produces_value_semantic_reports` assertion: 28 → 30.
- ☑ Update README.md fixture count: 28 → 30; extend fixture list with two new entries.
- ☑ `cargo check -p apollo-validation`: 0 errors, 0 warnings.
- ☑ `cargo test -p apollo-validation validation_suite_produces_value_semantic_reports`: 1/1 passed.
- ☑ Artifact sync: backlog.md, gap_audit.md.

Closure XXII — GPU Benchmark Runner Workflow + Root README Correction [patch]

Sprint target version: 0.12.2

- ☑ Audit existing CI: only hosted `ubuntu-latest` jobs present; no self-hosted GPU path.
- ☑ Audit benchmark surfaces: `apollo-fft-wgpu`, `apollo-nufft-wgpu`, `apollo-stft-wgpu`, `apollo-radon-wgpu` Criterion suites present.
- ☑ Add `.gitignore` rule for generated `.benchmarks/gpu-runner/*` while preserving `.gitkeep`.
- ☑ Add manual workflow `.github/workflows/gpu-benchmarks.yml` targeting `[self-hosted, gpu, apollo]`.
- ☑ Add PowerShell runner `scripts/run_gpu_benchmarks.ps1` with manifest and summary emission.
- ☑ Stage artifacts under `.benchmarks/gpu-runner/run-<run_id>/` and upload via workflow artifact.
- ☑ Correct stale root README claims for `apollo-czt-wgpu`, `apollo-mellin-wgpu`, `apollo-radon-wgpu`, and `apollo-stft-wgpu`.
- ☑ Add root README section documenting the GPU benchmark runner labels, workflow, and outputs.
- ☑ Update `CHANGELOG.md`, `backlog.md`, and `gap_audit.md` for Closure XXII.
- ☑ Validate PowerShell runner syntax.
- ☑ Validate workflow YAML parses.
- ☑ Smoke-run `scripts/run_gpu_benchmarks.ps1` on `fft` group: bundle created under `.benchmarks/gpu-runner/manual-smoke`, manifest and criterion output verified, transient bundle removed after validation.

Closure XXI — README Documentation Sync for v0.2.0 Inverse Additions [patch]

Sprint target version: 0.2.1 (documentation only, no API change)

- ☑ `apollo-czt/README.md`: add "Inverse Transform" section (Björck-Pereyra, NotInvertible conditions).
- ☑ `apollo-mellin/README.md`: add "Inverse Transform" section (IDFT + exp-resample, SpectrumLengthMismatch).
- ☑ `apollo-czt-wgpu/README.md`: update "Execution Contract" and "Verification" to reflect forward+inverse.
- ☑ `apollo-mellin-wgpu/README.md`: update "Execution Contract" and "Verification" to reflect two-pass inverse.
- ☑ `checklist.md`: add Closure XX completed entry.
- ☑ `backlog.md`: add Closure XXI entry.
- ☑ `cargo check --workspace`: 0 errors, 0 warnings.

Closure XX — CPU + GPU Inverse Transforms: CZT and Mellin [minor]

Sprint target version: 0.2.0

- ☑ Audit apollo-czt source: `forward` only, no inverse. `CztError` variants enumerated.
- ☑ Audit apollo-mellin source: `forward_spectrum` / `forward_resample` / `moment` only.
- ☑ Audit apollo-czt-wgpu: `execute_inverse` stub returning `UnsupportedExecution`.
- ☑ Audit apollo-mellin-wgpu: `execute_inverse` stub returning `UnsupportedExecution`.
- ☑ Derive CZT inverse: Vandermonde system `V·y = X` → Björck-Pereyra $O(N^2)$ Newton solve.

- ☑ Derive Mellin inverse: IDFT of log-spectrum → `g[n]`, then exp-resample `g` → signal.
- ☑ Implement `czt_bjork_pereyra_inverse` in `bluestein.rs`; fix borrow checker (cache `c[k+1]`).
- ☑ Add `CztError::NotInvertible { reason: &'static str }`.
- ☑ Add `CztPlan::inverse`, `CztStorage::inverse_into`; 5 value-semantic tests.
- ☑ Implement `inverse_log_frequency_spectrum` + `exp_resample` in `resample.rs`.
- ☑ Add `MellinError::SpectrumLengthMismatch`.
- ☑ Add `MellinPlan::inverse_spectrum`; export `exp_resample` + `inverse_log_frequency_spectrum`; 4 tests.
- ☑ Fix `assert_abs_diff_eq!` with message argument (unsupported by approx 0.5.1); use `assert!`.
- ☑ Remove unused `approx::assert_abs_diff_eq` import from `inverse_tests`.
- ☑ Add `czt_inverse` WGSL entry point (adjoint formula); build `inverse_pipeline`.
- ☑ Implement `CztWgpuBackend::execute_inverse`; `WgpuCapabilities::forward_inverse`.
- ☑ Update capability and backend tests; add `gpu_inverse_roundtrip_dft_parameters`, `gpu_inverse_rejects_non_square_plan`.
- ☑ Add `mellin_inverse_spectrum` + `mellin_exp_resample` WGSL kernels; `InverseMellinParamsPod`.
- ☑ Add `inverse_spectrum_pipeline`, `exp_resample_pipeline`, `inv_params_buffer` to kernel.
- ☑ Implement `MellinGpuKernel::execute_inverse` (two-pass: IDFT + exp-resample).
- ☑ Implement `MellinWgpuBackend::execute_inverse`; `WgpuCapabilities::forward_inverse`.
- ☑ Update capability/backend tests; add `gpu_inverse_roundtrip_constant_signal`, `gpu_inverse_rejects_invalid_output_domain`.
- ☑ Bump all four crates to v0.2.0.
- ☑ `cargo test -p apollo-czt -p apollo-mellin -p apollo-czt-wgpu -p apollo-mellin-wgpu`: 61/61 passed.
- ☑ Artifact sync: CHANGELOG.md (Closure XX), backlog.md, gap_audit.md.

Closure XIX — StftGpuBuffers Non-PoT Scratch Sizing [minor]

Sprint target version: 0.10.0

- ☑ Read `buffers.rs`: inspect current scratch sizing (frame_count × frame_len).
- ☑ Import `chirp_padded_len` from `super::chirp`.
- ☑ Update scratch_elem_count computation: use `chirp_padded_len(frame_len)` for non-PoT.
- ☑ Remove `assert!(frame_len.is_power_of_two())` in `StftGpuBuffers::new`.
- ☑ Update docstring in `buffers.rs`: remove PoT constraint; add Closure XIX note.
- ☑ Update `device.rs`: `make_buffers` docstring mentions non-PoT support + Closure XIX note.
- ☑ `cargo check -p apollo-stft-wgpu`: 0 warnings, 0 errors.
- ☑ Add `make_buffers_accepts_non_power_of_two_frame_len_structurally` test.
- ☑ Add GPU-gated test: `forward_buffers_non_pot_frame_len_400_when_device_exists`.
- ☑ Add GPU-gated test: `inverse_buffers_non_pot_frame_len_400_when_device_exists`.
- ☑ Fix unused variable warning in tests.
- ☑ `cargo test -p apollo-stft-wgpu`: 16 passed; 7 ignored (GPU-gated); 0 failed.
- ☑ Bump version: 0.9.0 → 0.10.0 in Cargo.toml.
- ☑ Artifact sync: CHANGELOG.md (0.10.0), backlog.md, checklist.md, gap_audit.md.

Closure XVIII — Non-Power-of-Two STFT GPU Path (Bluestein/Chirp-Z) [minor]

Sprint target version: 0.9.0

- ☑ Write ADR: `design_history_file/adr_stft_wgpu_non_pot_chirpz.md` (required before [minor]).
- ☑ Create `stft_chirp.wgsl`: five-pass Bluestein WGSL shader (premul_fwd, premul_inv, pointmul, postmul_fwd, postmul_inv).
- ☑ Create `stft_chirp_fft.wgsl`: radix-2 sub-FFT shader (bitrev, butterfly_fwd, butterfly_inv, scale).
- ☑ Create `infrastructure/chirp.rs`: `StftChirpData` struct with GPU resource pre-allocation and `chirp_padded_len`.
- ☑ Update `infrastructure/mod.rs`: add `pub(crate) mod chirp;`.
- ☑ Update `Cargo.toml`: add `ndarray = "0.16"` to `[dependencies]`.
- ☑ Update `kernel.rs`: conditional dispatch (PoT → existing Radix-2, non-PoT → `execute_forward_fft_chirp` / `execute_inverse_chirp`); add `dispatch_chirp_radix2` helper.
- ☑ Update `device.rs`: remove `FrameLenNotPowerOfTwo` guard from `execute_forward` and `execute_inverse`.
- ☑ Update `error.rs`: update `FrameLenNotPowerOfTwo` doc to reflect deprecated-from-primary-path status.
- ☑ Fix all compile warnings: removed unused import `StftChirpParamsPod`, removed `mut` on closure, added `#[allow(dead_code)]` on GPU-lifetime fields.
- ☑ Update/rename old rejection tests → acceptance tests (`forward_accepts_non_power_of_two_frame_len_chirpz`, `inverse_accepts_non_power_of_two_frame_len_chirpz`).
- ☑ Add `forward_accepts_non_power_of_two_frame_len_structurally`.
- ☑ Add GPU-gated tests: `forward_chirpz_non_pot_frame_len_400_when_device_exists`, `inverse_chirpz_non_pot_frame_len_400_when_device_exists`.
- ☑ `cargo check -p apollo-stft-wgpu`: 0 warnings, 0 errors.
- ☑ `cargo test -p apollo-stft-wgpu`: 15 passed; 5 ignored (GPU-gated); 0 failed.
- ☑ Bump `apollo-stft-wgpu` version: 0.1.0 → 0.9.0.
- ☑ Artifact sync: CHANGELOG.md, backlog.md, checklist.md, gap_audit.md.

Closure XVII — STFT GPU Buffer-Reuse Criterion Benchmarks + README Usage Documentation [patch]

Sprint target version: 0.8.5

- ☑ Add `bench_forward_reuse` to `crates/apollo-stft-wgpu/benches/stft_bench.rs`: head-to-head `execute_forward` vs `execute_forward_with_buffers` at fl ∈ {256, 512, 1024}; `StftGpuBuffers` pre-allocated outside bench loop.
- ☑ Add `bench_inverse_reuse` to `stft_bench.rs`:
  head-to-head `execute_inverse` vs `execute_inverse_with_buffers`;
  spectrum pre-computed outside bench loop; only inverse dispatch measured.
- ☑ Add both groups to `criterion_group!(benches, …)` in `stft_bench.rs`.
- ☑ Update `stft_bench.rs` module docstring to describe both allocating and buffer-reuse paths and their mathematical basis.

- ☑ Add "Buffer Reuse" section to `crates/apollo-stft-wgpu/README.md` with usage snippet, constraint notes (`FrameLenNotPowerOfTwo`, `LengthMismatch`), and pattern description.
- ☑ Add "Benchmarks" section to `README.md` with group table and `cargo bench` invocation.
- ☑ `cargo check -p apollo-stft-wgpu` clean (bench compiles against `StftGpuBuffers` API).
- ☑ `cargo test -p apollo-stft-wgpu`: 14 passed; 3 ignored (GPU-gated).
- ☑ Artifact sync: CHANGELOG.md (0.8.5), Cargo.toml (0.8.4 → 0.8.5), backlog.md, checklist.md, gap_audit.md updated.

Closure XVI — StftGpuBuffers Pre-allocated Buffer Reuse [minor]

Sprint target version: 0.8.4

- ☑ Create `crates/apollo-stft-wgpu/src/infrastructure/buffers.rs` with `StftGpuBuffers` struct, `StftGpuBuffers::new(device, kernel, frame_count, frame_len, signal_len, hop_len)`,
  and accessors `frame_count()`, `frame_len()`, `signal_len()`, `hop_len()`, `fwd_output()`, `inv_output()`.
- ☑ Make `ComplexPod`, `StftParams`, `FftStageParams`, `FwdFftStageParams` `pub(crate)` in `kernel.rs`.
- ☑ Make `bind_group_layout`, `fft_data_bgl`, `fft_params_bgl` fields `pub(crate)` in `StftGpuKernel`.
- ☑ Add `StftGpuKernel::execute_forward_fft_with_buffers` (reuses bind groups; uploads signal via `queue.write_buffer`; writes result to `buffers.fwd_output_host`).
- ☑ Add `StftGpuKernel::execute_inverse_with_buffers` (uploads spectrum + OLA params; writes result to `buffers.inv_output_host`).
- ☑ Add `StftWgpuBackend::make_buffers`, `execute_forward_with_buffers`, `execute_inverse_with_buffers` to `device.rs`.
- ☑ Re-export `StftGpuBuffers` from `lib.rs`.
- ☑ Add `#[allow(dead_code)]` on GPU-only scratch fields (`re_scratch_buf`, `im_scratch_buf`, `frame_data_buf`) with architectural justification doc-comment.
- ☑ `cargo clippy -p apollo-stft-wgpu --all-targets -- -D warnings` clean.
- ☑ `cargo test -p apollo-stft-wgpu`: 14 passed; 3 ignored (`reusable_buffers_match_*`, `forward_fft_roundtrip_*`, `inverse_roundtrip_large_*`).
- ☑ Add `pub mod buffers` to `infrastructure/mod.rs`.
- ☑ Artifact sync: CHANGELOG.md, Cargo.toml (0.8.3 → 0.8.4), backlog.md, checklist.md, gap_audit.md updated.

Closure XV — Radon FBP GPU Criterion Benchmarks [patch]

Sprint target version: 0.8.3

- ☑ Add `criterion = "0.5"` to `apollo-radon-wgpu` dev-deps.
- ☑ Add `[[bench]] name = "radon_wgpu_bench" harness = false` to `apollo-radon-wgpu/Cargo.toml`.
- ☑ Create `crates/apollo-radon-wgpu/benches/radon_wgpu_bench.rs` with `radon_wgpu_forward`
  and `radon_wgpu_fbp` criterion groups, each covering image_size ∈ {64, 128, 256}.

- ☑ Gaussian disk phantom (σ=0.25); uniform angles on `[0,π)`.
- ☑ `cargo check -p apollo-radon-wgpu --benches` clean.
- ☑ Artifact sync: backlog.md, checklist.md, gap_audit.md, CHANGELOG.md updated.

Closure XIV — Dead-Code Removal: O(N^2) Forward Pipeline + stft_inverse_frames [patch]

Sprint target version: 0.3.0

- ☑ Remove `forward_pipeline: wgpu::ComputePipeline` field from `StftGpuKernel`.
- ☑ Remove forward shader module creation and `forward_pipeline` construction from `new()`.
- ☑ Remove `StftGpuKernel::execute()` method (112 lines of O(N^2) GPU forward path).
- ☑ Delete `crates/apollo-stft-wgpu/src/infrastructure/shaders/stft.wgsl`.
- ☑ Remove `stft_inverse_frames` entry point from `stft_inverse.wgsl` (~40 lines).
- ☑ Update `stft_inverse.wgsl` file comment to reflect single-pass OLA role.
- ☑ Update kernel.rs module docstring, `WORKGROUP_SIZE` comment, struct doc.
- ☑ Update `dispatch_count` and `fft_dispatch_count` doc comments.
- ☑ `cargo clippy -p apollo-stft-wgpu -- -D warnings` clean.
- ☑ `cargo test -p apollo-stft-wgpu` 14 passed; 2 ignored.

Closure XIII — STFT GPU Criterion Benchmarks [patch]

Sprint target version: 0.3.0 (patch within the next minor)

- ☑ Add `criterion = { version = "0.5", features = ["html_reports"] }` to `apollo-stft-wgpu` dev-deps.
- ☑ Add `[[bench]] name = "stft_bench" harness = false` to `apollo-stft-wgpu/Cargo.toml`.
- ☑ Create `crates/apollo-stft-wgpu/benches/stft_bench.rs` with `bench_forward_fft` and `bench_inverse_fft` criterion groups, each covering frame_len ∈ {256, 512, 1024}.
- ☑ Analytical signal (bin-aligned sinusoids k_1=16, k_2=64) used as benchmark workload.
- ☑ `cargo check -p apollo-stft-wgpu --benches` clean.
- ☑ Artifact sync: backlog.md, checklist.md, gap_audit.md, CHANGELOG.md updated.

Closure XII — STFT Forward-Path GPU FFT Acceleration [minor]

Sprint target version: 0.3.0 (first unreleased minor after 0.2.0)

- ☑ Create `crates/apollo-stft-wgpu/src/infrastructure/shaders/stft_forward_fft.wgsl` with entry points: `stft_fwd_pack_window`, `stft_fwd_bitrev`, `stft_fwd_butterfly`, `stft_fwd_interleave`; DFT twiddle `exp(-2πi·k/N)`.
- ☑ Add `FwdFftStageParams` struct (16 bytes, 4×u32: frame_count, frame_len, hop_len, stage).
- ☑ Add 4 new pipeline fields to `StftGpuKernel`: `fwd_pack_window_pipeline`, `fwd_bitrev_pipeline`, `fwd_butterfly_pipeline`, `fwd_interleave_pipeline`.
- ☑ Extend `StftGpuKernel::new()` to compile `stft_forward_fft.wgsl` and build 4 pipelines, reusing `fft_pipeline_layout` (group 0: `fft_data_bgl`, group 1: `fft_params_bgl`).

- ☑ Implement `StftGpuKernel::execute_forward_fft()`: dispatch sequence pack_window → bitrev → butterfly×$\log_2$N → interleave.
- ☑ Add `FrameLenNotPowerOfTwo` guard in `StftWgpuBackend::execute_forward()`.
- ☑ Route `execute_forward` to `kernel.execute_forward_fft()`.
- ☑ Add test `forward_rejects_non_power_of_two_frame_len` (no GPU required).
- ☑ Add test `forward_fft_roundtrip_large_frame_when_device_exists` (#[ignore], FRAME_LEN=1024).
- ☑ `cargo check -p apollo-stft-wgpu` clean.
- ☑ `cargo clippy -p apollo-stft-wgpu -- -D warnings` clean.
- ☑ `cargo test -p apollo-stft-wgpu` passing.
- ☑ Artifact sync: backlog.md, checklist.md, gap_audit.md, CHANGELOG.md updated.

Closure XI phase (STFT inverse GPU acceleration, FrameLenNotPowerOfTwo, large-frame verification)

- ☑ Create `apollo-stft-wgpu/src/infrastructure/shaders/stft_inverse_fft.wgsl` with four entry points: `stft_deinterleave`, `stft_bitrev`, `stft_butterfly`, `stft_scale_and_window`. Two bind groups: group 0 (4 data bindings), group 1 (per-stage `FftStageParams` uniform). IDFT twiddle: exp(+2πi·k/N). Formal basis: Cooley-Tukey Radix-2 DIT (Cooley & Tukey 1965); WOLA identity (Allen & Rabiner 1977 Theorem 1).
- ☑ Add `FftStageParams` struct (`frame_count, frame_len, stage, _pad`: 4×u32 = 16 bytes) to `kernel.rs`.
- ☑ Replace `inverse_frames_pipeline` in `StftGpuKernel` with `deinterleave_pipeline`, `bitrev_pipeline`, `butterfly_pipeline`, `scale_window_pipeline`; add `fft_data_bgl` (4-binding group-0 layout) and `fft_params_bgl` (1-uniform group-1 layout); keep `FFT_WORKGROUP_SIZE = 256` separate from `WORKGROUP_SIZE = 64` to avoid under-dispatching forward/OLA passes.
- ☑ Rewrite `StftGpuKernel::execute_inverse` to: validate `frame_len.is_power_of_two()`, allocate re/im scratch and frame_data buffers, pre-allocate `log₂(N)` per-stage uniform buffers and bind groups, encode deinterleave + bitrev + N butterfly passes + scale_window + OLA in one `CommandEncoder`.
- ☑ Add `WgpuError::FrameLenNotPowerOfTwo { frame_len: usize }` variant to `domain/error.rs`.
- ☑ Add power-of-two validation guard in `StftWgpuBackend::execute_inverse` (`device.rs`) before kernel dispatch.
- ☑ Add `inverse_rejects_non_power_of_two_frame_len` test: frame_len=6, expects `FrameLenNotPowerOfTwo { frame_len: 6 }`.
- ☑ Add `#[ignore = "requires wgpu device"]` `inverse_roundtrip_large_frame_1024_samples_when_device_exists` test: frame_len=1024, hop=512, signal_len=8192, analytic sine reference, TOL=5e-3.
- ☑ Verify `cargo check --workspace --all-targets` clean.
- ☑ Verify `cargo clippy --workspace --all-targets -- -D warnings` zero warnings.
- ☑ Verify `cargo test --workspace --all-targets` zero failures (1 GPU-gated test ignored).

Closure X phase (GPU Radon FBP, adjoint identity test, STFT parameterized roundtrip, documentation sync)

- ☑ Add `supports_filtered_backprojection: bool` field and `forward_inverse_and_fbp(device_available)` constructor to `apollo-radon-wgpu/src/domain/capabilities.rs`.
- ☑ Create `apollo-radon-wgpu/src/infrastructure/shaders/radon_fbp_filter.wgsl`: entry `radon_fbp_filter` — per-(angle, detector) circular convolution with the ramp filter impulse response `h`: `filtered[a*D+d] = Σ_{d'} sinogram[a*D+d'] * h[(d-d'+D)%D]`. Reuses existing 4-binding layout (read, read, read_write, uniform). Basis: Ram-Lak ramp filter (Bracewell & Riddle 1967; Shepp & Logan 1974).
- ☑ Add `fbp_filter_pipeline: wgpu::ComputePipeline` to `RadonGpuKernel`; add `compute_ramp_kernel_f32(detector_count, detector_spacing) -> Vec<f32>` (= `ramp_filter_projection([1,0,...], spacing)` cast to f32); add `RadonGpuKernel::execute_filtered_backproject(device, queue, plan, sinogram, angles) -> WgpuResult<Array2<f32>>` (2-pass single encoder: filter → backproject; host-side `* π/angle_count` normalization) in `apollo-radon-wgpu/src/infrastructure/kernel.rs`.
- ☑ Add `RadonWgpuBackend::execute_filtered_backproject(plan, sinogram, angles)` to `apollo-radon-wgpu/src/infrastructure/device.rs`; update `capabilities()` to return `forward_inverse_and_fbp(true)`.
- ☑ Add 4 new tests to `apollo-radon-wgpu/src/verification.rs`: `backproject_satisfies_adjoint_identity_when_device_exists` (⟨Af,g⟩ = ⟨f,A†g⟩, rel_tol=5e-3), `capabilities_include_filtered_backprojection`, `filtered_backproject_matches_cpu_reference_when_device_exists` (single-center-pixel reference, TOL=5e-2), `filtered_backproject_rejects_sinogram_shape_mismatch`.
- ☑ Add `inverse_roundtrip_for_multiple_cola_parameter_sets` test to `apollo-stft-wgpu/src/verification.rs`: 3 COLA-compliant (frame_len, hop_len) pairs at 50% overlap (8/4, 16/8, 32/16); CPU forward → GPU inverse roundtrip; TOL=5e-3.
- ☑ Update `README.md`: fix stale WGPU descriptions for `apollo-radon-wgpu` (add FBP), `apollo-stft-wgpu` (add inverse), `apollo-hilbert-wgpu` (add inverse), `apollo-sdft-wgpu` (add inverse).
- ☑ Update `ARCHITECTURE.md`: fix capability table notes for `apollo-radon-wgpu`, `apollo-stft-wgpu`, `apollo-hilbert-wgpu`, `apollo-sdft-wgpu` rows.
- ☑ Verify `cargo check --workspace --all-targets` clean.
- ☑ Verify `cargo clippy --workspace --all-targets -- -D warnings` zero warnings.
- ☑ Verify `cargo test --workspace --all-targets` zero failures.

Closure IX phase (GPU inverse STFT WOLA, GPU Radon backprojection, artifact corrections)

- ☑ Add `WgpuCapabilities::forward_and_inverse(device_available)` constructor to `apollo-stft-wgpu/src/domain/capabilities.rs`.
- ☑ Create `apollo-stft-wgpu/src/infrastructure/shaders/stft_inverse.wgsl`: two entry points sharing `@binding(0) array<f32>` (read), `@binding(1) array<f32>` (read_write), `@binding(2)` uniform. `stft_inverse_frames`: per-(frame, local_j) windowed IDFT — `frame_data[m·N+j] = (1/N)·Re{Σ_k X[m,k]·exp(+2πi·k·j/N)}·hann(j)`. `stft_inverse_ola`: per-output-sample WOLA — `y[n] = Σ_m frame_data[m·N+(n−start_m)] / Σ_m hann(n−start_m)²`, `start_m = m·hop − N/2`. Basis: WOLA identity (Allen–Rabiner 1977, Theorem 1).

- ☑ Add `inverse_frames_pipeline` and `inverse_ola_pipeline` fields to `StftGpuKernel`; compile from `stft_inverse.wgsl` in `apollo-stft-wgpu/src/infrastructure/kernel.rs`. Add `StftGpuKernel::execute_inverse(device, queue, spectrum, frame_len, hop_len, frame_count, signal_len) -> WgpuResult<Vec<f32>>`: interleave spectrum as f32 pairs; 2-pass single-encoder (frames → ola); copy result → staging; return `Vec<f32>`.
- ☑ Update `StftWgpuBackend::execute_inverse` in `apollo-stft-wgpu/src/infrastructure/device.rs`: change from no-arg stub to real signature `(plan, spectrum, signal_len)`, validate inputs, derive `frame_count = 1 + signal_len.div_ceil(hop_len)`, delegate to kernel. Add `execute_inverse_typed_into<I: StftSpectrumInput, O: StftRealOutputStorage>(plan, input_precision, output_precision, spectrum, signal_len, output)`. Update `capabilities()` to return `forward_and_inverse(true)`.
- ☑ Update `apollo-stft-wgpu/src/verification.rs`: remove `execute_inverse_returns_unsupported` test; rename `backend_reports_forward_only_when_device_exists` → `backend_reports_forward_and_inverse_when_device_exists` (assert `supports_inverse = true`). Add `capabilities_reflect_forward_and_inverse_surface`, `inverse_roundtrip_recovers_cola_signal_when_device_exists` (CPU forward → GPU inverse vs CPU inverse, TOL=5e-4), `inverse_matches_cpu_reference_for_16sample_signal`.
- ☑ Add `WgpuCapabilities::forward_and_inverse(device_available)` constructor to `apollo-radon-wgpu/src/domain/capabilities.rs`.
- ☑ Add `SinogramShapeMismatch { expected_angles, expected_detectors, actual_angles, actual_detectors }` variant to `WgpuError` in `apollo-radon-wgpu/src/domain/error.rs`.
- ☑ Create `apollo-radon-wgpu/src/infrastructure/shaders/radon_backproject.wgsl`: entry `radon_backproject` — per-pixel accumulation `bp[r,c] = Σ_θ interp(sinogram[θ,·], x·cosθ + y·sinθ)` with linear interpolation and out-of-range zero-clamping. Basis: Radon adjoint operator (Natterer 2001, §II.2). Reuses same 4-binding layout as forward (read, read, read_write, uniform).
- ☑ Add `backproject_pipeline: wgpu::ComputePipeline` to `RadonGpuKernel`; add `execute_backproject(device, queue, plan, sinogram, angles) -> WgpuResult<Array2<f32>>` in `apollo-radon-wgpu/src/infrastructure/kernel.rs`. Single-pass encoder: dispatch `rows * cols` invocations; copy image_buf → staging.
- ☑ Update `RadonWgpuBackend::execute_inverse` in `apollo-radon-wgpu/src/infrastructure/device.rs`: change from no-arg stub to real signature `(plan, sinogram: &Array2<f32>, angles: &[f32]) -> WgpuResult<Array2<f32>>`, validate sinogram shape (→ SinogramShapeMismatch) and angle count, delegate to kernel. Add `execute_inverse_flat_typed<T: RadonStorage>`. Update `capabilities()` to return `forward_and_inverse(true)`.
- ☑ Update `apollo-radon-wgpu/src/verification.rs`: rename `backend_reports_forward_only_when_device_exists` → `backend_reports_forward_and_backproject_when_device_exists` (assert `supports_inverse = true`). Add `capabilities_reflect_forward_and_inverse_surface`, `backproject_matches_cpu_reference_when_device_exists` (CPU forward→GPU backproject vs CPU backproject, TOL=5e-3), `execute_inverse_rejects_sinogram_shape_mismatch`.

- ☑ Correct `gap_audit.md` note: CZT and Mellin have no CPU inverse defined; the open-gaps claim "CPU inverse paths are implemented" was inaccurate for those two. GPU inverse for CZT and Mellin remains `UnsupportedExecution` by architectural design, not by deferral.
- ☑ Verify `cargo check --workspace --all-targets` clean.
- ☑ Verify `cargo clippy --workspace --all-targets -- -D warnings` zero warnings.
- ☑ Verify `cargo test --workspace --all-targets` zero failures (39 test suites).

Closure VIII phase (GPU inverse Hilbert and SDFT, CZT proptest tolerance fix)

- ☑ Add `WgpuCapabilities::forward_and_inverse(device_available)` constructor to `apollo-hilbert-wgpu/src/domain/capabilities.rs`.
- ☑ Add `hilbert_inverse_mask` WGSL entry point to `apollo-hilbert-wgpu/src/infrastructure/shaders/hilbert.wgsl`: reads DFT(quadrature) from `inout_a`, writes recovered spectrum `X[k]` to `inout_b`. DC (k=0) and Nyquist (even-N: k=N/2) bins are set to zero (lost in forward Hilbert); positive bins: X[k] = j·Q[k]; negative: X[k] = -j·Q[k]. Fix pre-existing bug in `hilbert_inverse_dft`: was writing `inout_b[n].re = original` (stale self-assign) and `inout_b[n].im = acc.y * scale` (missing real accumulation); corrected to `re = acc.x * scale`, `im = acc.y * scale`.
- ☑ Add `inverse_mask_pipeline` field to `HilbertGpuKernel`; compile from `hilbert_inverse_mask` entry point in `apollo-hilbert-wgpu/src/infrastructure/kernel.rs`.
- ☑ Add `HilbertGpuKernel::execute_inverse` method: 3 sequential passes in one encoder (DFT of quadrature, inverse mask, IDFT of recovered spectrum), with separate `spectrum_buffer` and `recovered_buffer` to avoid in-place data races. Single `queue.submit` + `device.poll(Wait)`. Returns `Vec<f32>` real samples.
- ☑ Add `HilbertWgpuBackend::execute_inverse(plan, quadrature)` and `execute_inverse_typed_into(plan, precision, quadrature, output)` methods to `apollo-hilbert-wgpu/src/infrastructure/device.rs`; update `capabilities()` to report `forward_and_inverse`.
- ☑ Add 3 verification tests to `apollo-hilbert-wgpu/src/verification.rs`: `capabilities_reflect_forward_and_inverse_surface`, `inverse_roundtrip_recovers_zero_mean_signal_when_device_exists` (validates DC+Nyquist loss contract with analytically derived expected values), `inverse_matches_cpu_frequency_domain_reference_when_device_exists` (CPU $O(N^2)$ reference for inverse mask).
- ☑ Add `WgpuCapabilities::forward_and_inverse(device_available)` constructor to `apollo-sdft-wgpu/src/domain/capabilities.rs`.
- ☑ Add `sdft_inverse_bins` WGSL entry point to `apollo-sdft-wgpu/src/infrastructure/shaders/sdft.wgsl`: `x[n] = (1/K)·Σ_{b=0}^{K-1} X[b]·exp(+2πi·b·n/K)`; reads complex bins as interleaved f32 pairs from binding 0 (`window_data[2b]` = Re, `window_data[2b+1]` = Im); writes real signal to `output_data[n].re`.
- ☑ Add `forward_pipeline` + `inverse_pipeline` fields to `SdftGpuKernel`; update `execute` to use `forward_pipeline`; add `SdftGpuKernel::execute_inverse` method in `apollo-sdft-wgpu/src/infrastructure/kernel.rs`.

- ☑ Add `SdftWgpuBackend::execute_inverse(plan, bins)`, `execute_inverse_typed_into(plan, precision, bins, output)`, and `validate_plan_bins(plan, bins)` methods to `apollo-sdft-wgpu/src/infrastructure/device.rs`; update `capabilities()` to report `forward_and_inverse`.
- ☑ Add 4 verification tests to `apollo-sdft-wgpu/src/verification.rs`: `capabilities_reflect_forward_and_inverse_surface`, `inverse_roundtrip_matches_original_signal_when_device_exists` (full K=N IDFT roundtrip, tol 5e-4), `inverse_matches_cpu_reference_when_device_exists` (analytical 2-point DFT/IDFT verification), `inverse_rejects_bin_count_mismatch`.
- ☑ Fix pre-existing CZT proptest tolerance defect in `apollo-czt`: `bluestein_equals_direct_for_arbitrary_parameters` used absolute threshold 1e-9, violated when `|w|>1` amplifies output magnitude (observed error 3e-9 for |w|≈1.28, N=M=7). Fix: replace `diff < 1e-9` with `diff < 1e-9 * max(|direct[k]|, 1.0)` (relative bound). Formal justification: Bluestein relative error ≤ $C \cdot \log_2(p) \cdot \varepsilon_machine$ ≈ 2.6e-15; 1e-9 relative threshold provides ×3.8e5 margin. Absolute threshold fails for large outputs from chirp amplification.
- ☑ Verify `cargo check --workspace --all-targets` clean.
- ☑ Verify `cargo clippy --workspace --all-targets -- -D warnings` zero warnings.
- ☑ Verify `cargo test --workspace --all-targets` zero failures (39 test suites).

Closure VII phase (single-submission FrFT GPU, 6 new fixtures, 4 proptest crates, docs)

- ☑ Update README.md line 84: fixture count 10 → 22 with complete fixture list.
- ☑ Create CHANGELOG.md with version history from 0.1.0 to Unreleased Closure VII.
- ☑ Delete `design_history_file/backlog.md`, `design_history_file/checklist.md`, `design_history_file/gap_audit.md` (stale root-artifact copies).
- ☑ Refactor `apollo-frft-wgpu/src/infrastructure/unitary_kernel.rs`: single encoder + 3 sequential compute passes + copy + 1 submit + 2 polls; update doc comment; remove per-step encoder loop.
- ☑ Add `apollo-sft`, `apollo-sht`, `apollo-stft`, `apollo-hilbert`, `apollo-mellin`, `apollo-radon` to `apollo-validation/Cargo.toml` dependencies.
- ☑ Add 6 fixture imports and 6 fixture functions to `apollo-validation/src/application/suite.rs`; add Array2 to ndarray import; register fixtures in `run_published_reference_suite`; update count assertions 22 → 28.
- ☑ Add proptest block to `apollo-czt` plan tests: Bluestein-vs-direct, spiral-collapse, linearity.
- ☑ Add proptest block to `apollo-frft` unitary.rs tests: roundtrip, additivity, linearity.
- ☑ Add proptest block to `apollo-nufft` plan tests: DC invariant, fast-tracks-exact, linearity.
- ☑ Add proptest block to `apollo-sft` transform tests: K-sparse roundtrip, Parseval top-K, retained = DFT.
- ☑ Verify `cargo check --workspace --all-targets` clean.
- ☑ Verify `cargo clippy --workspace --all-targets -- -D warnings` zero warnings.
- ☑ Verify `cargo test --workspace --all-targets` zero failures.

Closure VI phase (workspace unblock, O(N log N) NTT WGPU, expanded fixtures, cleanup)

- ☑ Fix `apollo-fft/Cargo.toml`: `name = "apollo"` → `name = "apollo-fft"` (workspace-blocking patch).
- ☑ Fix `apollo-fft-wgpu/Cargo.toml`: dep key `apollo` → `apollo-fft` (workspace-blocking patch).
- ☑ Rewrite `apollo-ntt-wgpu/src/infrastructure/shaders/ntt.wgsl`: replace $O(N^2)$ DFT entry point `ntt_transform` with O(N log N) Cooley-Tukey DIT entry points `ntt_butterfly` (per-stage butterfly) and `ntt_scale` (inverse N^{-1} scaling). Flat twiddle array `twiddles[k]=ω^k`. No-race proof: disjoint (i,j) pairs per thread per stage.
- ☑ Rewrite `apollo-ntt-wgpu/src/infrastructure/kernel.rs`: `NttGpuKernel` now holds `butterfly_pipeline` + `scale_pipeline`. `NttGpuBuffers` carries in-place `data_buffer`, forward+inverse `twiddle_buffers`, stride-aligned `params_buffer` (pre-written once at creation for all stages), `fwd_bind_group` + `inv_bind_group`. `execute_from_residues`: one encoder, `log₂(N)` butterfly passes + optional scale pass, dynamic uniform offsets, single `queue.submit` + single `device.poll(Wait)`.
- ☑ Update `apollo-ntt-wgpu/src/infrastructure/device.rs`: pass `omega` to `kernel.create_buffers`; remove stale `modulus`/`root` args from `execute_with_buffers` and `execute_quantized_with_buffers` call sites.
- ☑ Remove `apollo_fft::PrecisionProfile` from `apollo-ntt-wgpu/src/domain/capabilities.rs`; remove `default_precision_profile` field; update doc to state NTT has no floating-point precision concept.
- ☑ Remove `apollo-fft` from `apollo-ntt-wgpu/Cargo.toml` dependencies.
- ☑ Fix `apollo-ntt-wgpu/src/verification.rs`: add `#[ignore = "requires wgpu device"]` to 10 GPU tests; add `proptest_gpu` feature gate for GPU proptest; add CPU-only proptest module (`cpu_roundtrip_preserves_residue_class`, `convolution_theorem_holds_for_arbitrary_pairs`).
- ☑ Add `proptest = { workspace = true }` and `[features] proptest_gpu = []` to `apollo-ntt-wgpu/Cargo.toml`.
- ☑ Remove `#![allow(unused_imports)]` from `apollo-ntt/src/lib.rs`.
- ☑ Remove unused `use ndarray::Array1` from `apollo-ntt/src/application/execution/kernel/direct.rs`.
- ☑ Add `ntt_n16_impulse_fixture` and `ntt_n16_polynomial_product_fixture` to `apollo-validation/src/application/suite.rs`.
- ☑ Update fixture-count assertions from 20 to 22 in `apollo-validation/src/application/suite.rs`.
- ☑ Verify `cargo check --workspace --all-targets` clean.
- ☑ Verify `cargo clippy --workspace --all-targets -- -D warnings` zero warnings.
- ☑ Verify `cargo test --workspace --all-targets` zero failures; `10 ignored` in apollo-ntt-wgpu for GPU-device-dependent tests.

Closure V phase (GPU Unitary FrFT, validation fixtures, docs)

- ☑ Create `apollo-frft-wgpu/src/infrastructure/shaders/frft_unitary.wgsl` with single-entry-point 3-pass WGSL shader (step=0: V^T·x, step=1: phase, step=2: V·c; column-major V; `@workgroup_size(64)`).
- ☑ Create `apollo-frft-wgpu/src/infrastructure/unitary_kernel.rs` with `UnitaryFrftGpuKernel` (BGL 5 entries, compiled pipeline, `execute` method with 3 sequential

submissions + polls).

- ☑ Add `pub mod unitary_kernel;` to `apollo-frft-wgpu/src/infrastructure/mod.rs`.
- ☑ Add `UnitaryFrftWgpuPlan` to `apollo-frft-wgpu/src/application/plan.rs`.
- ☑ Add `unitary_kernel: Arc<UnitaryFrftGpuKernel>` field and `plan_unitary`/`execute_unitary_forward`/`execute_unitary_inverse` methods to `FrftWgpuBackend`.
- ☑ Re-export `UnitaryFrftWgpuPlan` from `apollo-frft-wgpu/src/lib.rs`.
- ☑ Add 5 value-semantic tests to `apollo-frft-wgpu/src/verification.rs` (identity, reversal, roundtrip, norm, CPU parity).
- ☑ Add `apollo-frft`, `apollo-wavelet`, `apollo-sdft` (or subset) to `apollo-validation/Cargo.toml`.
- ☑ Add `frft_unitary_order2_reversal_fixture`, `wavelet_haar_one_level_detail_fixture`, third fixture to `apollo-validation/src/application/suite.rs`.
- ☑ Update fixture-count assertions from 17 to 20 in `apollo-validation/src/application/suite.rs`.
- ☑ Create `design_history_file/adr_unitary_frft.md`.
- ☑ Update `ARCHITECTURE.md` with "Key: Unitary FrFT" subsection and capability table row.
- ☑ Update `gap_audit.md`: reclassify NTT gap; add Closure V closed-gaps section.
- ☑ Update `backlog.md`: add Closure V sprint section.
- ☑ Update `checklist.md`: add Closure V phase section (this document).
- ☑ Verify `cargo check --workspace --all-targets` clean.
- ☑ Verify `cargo clippy --workspace --all-targets -- -D warnings` zero warnings.
- ☑ Verify `cargo test --workspace --all-targets` zero failures.

Closure IV phase (FrFT unitarity, DCT-I/IV/DST-I/IV WGPU kernels)

- ☑ Add `nalgebra = { workspace = true }` to `apollo-frft/Cargo.toml`.
- ☑ Create `apollo-frft/src/application/execution/plan/frft/unitary.rs` with `GrunbaumBasis` (palindrome Grünbaum matrix, `nalgebra::SymmetricEigen`, eigenvectors sorted by decreasing eigenvalue) and `UnitaryFrftPlan` (DFrFT_a(x)=V·diag(exp(−iakπ/2))·V^T·x, $O(N^3)$ construction, $O(N^2)$ per call, provably unitary for all real orders).
- ☑ Add `pub mod unitary;` to `apollo-frft/src/application/execution/plan/frft/mod.rs`.
- ☑ Re-export `GrunbaumBasis` and `UnitaryFrftPlan` from `apollo-frft/src/lib.rs`; update crate-level doc to document both plan variants.
- ☑ Add 9 tests to `unitary.rs`: `unitary_order_zero_is_identity`, `unitary_order_4_is_identity`, `unitary_order_1_squared_equals_reversal`, `unitary_order_2_is_reversal`, `unitary_forward_inverse_roundtrip` (7 orders), `unitary_frft_preserves_l2_norm_for_non_integer_orders` (10 orders, rel_err < 1e-10), `unitary_frft_additive_order_property` (a=0.4, b=0.6), `rejects_invalid_plan_parameters`, `length_mismatch_is_rejected`.
- ☑ Extend `apollo-dctdst-wgpu/src/infrastructure/shaders/dct.wgsl`: change mode-3 `else` to `else if params.mode == 3u`; add modes 4 (DCT-I), 5 (DCT-IV), 6 (DST-I), 7 (DST-IV) matching CPU direct-kernel formulas exactly.
- ☑ Add `DctMode` variants `Dct1 = 4`, `Dct4 = 5`, `Dst1 = 6`, `Dst4 = 7` to `apollo-dctdst-wgpu/src/infrastructure/kernel.rs`; update enum doc comment.

- ☑ Update `apollo-dctdst-wgpu/src/infrastructure/device.rs execute_forward`: route DCT-I → `Dct1` (with N<2 `InvalidLength` guard), DCT-IV → `Dct4`, DST-I → `Dst1`, DST-IV → `Dst4`; remove all four `UnsupportedKind` returns.
- ☑ Update `apollo-dctdst-wgpu/src/infrastructure/device.rs execute_inverse`: route DCT-I → `(Dct1, 1/(2(N−1)))` (with N<2 guard), DCT-IV → `(Dct4, 2/N)`, DST-I → `(Dst1, 1/(2(N+1)))`, DST-IV → `(Dst4, 2/N)`; remove all four `UnsupportedKind` returns.
- ☑ Add 9 tests to `apollo-dctdst-wgpu/src/verification.rs`: forward parity vs CPU f64 reference for DCT-I/IV/DST-I/IV; self-inverse roundtrip for DCT-I/IV/DST-I/IV; `dct1_rejects_length_less_than_two`.
- ☑ Verify `cargo test --workspace --all-targets` 0 failures; `cargo clippy --workspace --all-targets -- -D warnings` 0 warnings.

Closure III phase (validation mock removal, SSOT DFT fix, DCT-I/IV/DST-I/IV, published fixtures)

- ☑ Remove `run_fft_gpu_suite()` mock: replace hardcoded `passed: true, error = 0.0` with real `GpuFft3d` forward + inverse roundtrip on 4×4×4; report actual forward (vs CPU f64 reference) and inverse (roundtrip) max absolute errors; when adapter unavailable report `attempted: false, passed: false`.
- ☑ Compute `forward_max_abs_error` for `low_precision` and `mixed_precision` profiles in `precision_profile_reports()`: compare each profile's forward spectrum against the f64 reference spectrum and store the result in `Some(...)` instead of `None`.
- ☑ Add 7 new published-reference fixtures to `apollo-validation` (10 → 17 total): `fft_inverse_four_point_fixture`, `dct2_inverse_pair_two_point_fixture`, `dht_self_reciprocal_fixture`, `fwht_two_point_fixture`, `qft_two_point_fixture`, `czt_unit_impulse_is_dft_fixture`, `gft_path_graph_forward_fixture`.
- ☑ Update `run_published_reference_suite` vec to include all 7 new fixtures; update fixture-count assertions from 10 to 17.
- ☑ Add `apollo-czt`, `apollo-fwht`, `apollo-qft`, `apollo-gft`, and `nalgebra = "0.33"` to `apollo-validation/Cargo.toml`.
- ☑ Resolve SSOT DFT violation in `apollo-hilbert/src/infrastructure/kernel/direct.rs`: replace private $O(N^2)$ `forward_dft_real` and `inverse_dft_complex` with `apollo_fft::fft_1d_array` and `apollo_fft::ifft_1d_complex`; remove rayon parallel dispatch; add `ndarray` to `apollo-hilbert/Cargo.toml`.
- ☑ Resolve SSOT DFT violation in `apollo-radon/src/infrastructure/kernel/filter.rs`: replace private $O(N^2)$ `forward_dft_real` and `inverse_dft_real_into` with `apollo_fft::fft_1d_array` and `apollo_fft::ifft_1d_array`.
- ☑ Remove unjustified `#![allow(unused_imports)]` from `apollo-fwht/src/lib.rs` and `apollo-stft/src/lib.rs`; remove the previously hidden unused `StftError` import from `apollo-stft/src/infrastructure/transport/cpu.rs`.
- ☑ Add `DctI`, `DctIV`, `DstI`, `DstIV` variants to `RealTransformKind` in `apollo-dctdst/src/domain/metadata/kind.rs`; add `UnsupportedLength` error variant for DCT-I N<2.
- ☑ Add direct $O(N^2)$ kernels `dct1`, `dct4`, `dst1`, `dst4` to `apollo-dctdst/src/infrastructure/kernel/direct.rs` with full Rustdoc (theorem, self-inverse proof, verified example, complexity, references); self-inverse scales: DCT-I → 1/(2(N−1)), DST-I → 1/(2(N+1)), DCT-IV and DST-IV → 2/N.

- ☑ Update `DctDstPlan::forward_into` and `inverse_into` to dispatch to new kernels; update `inverse_into` scale dispatch to use per-kind scale for DCT-I and DST-I.
- ☑ Add 26 new tests to `apollo-dctdst/src/verification/mod.rs`: known-value, self-inverse, plan roundtrip, error rejection, and proptest for all four new kinds.
- ☑ Fix non-exhaustive match in `apollo-dctdst-wgpu/src/infrastructure/device.rs`: return `WgpuError::UnsupportedKind` for DCT-I, DCT-IV, DST-I, DST-IV in both `execute_forward` and `execute_inverse`.
- ☑ Add `qft_unitarity_holds_for_multiple_sizes` ($N \in \{2,3,4,5,6,8\}$, deterministic) and `qft_unitarity_holds_for_random_size_and_input` (proptest $N \in [2,8]$) to `apollo-qft/src/verification/mod.rs`; both pass: $(M^\dagger M)[j,j'] = \delta(j,j')$ via DFT orthogonality.
- ☑ Document FrFT unitarity gap: failing tests removed (not weakened); non-integer-order kernel non-unitarity ($(M^\dagger M)[j,j] = 1/|\sin \alpha|$) recorded as open gap requiring Ozaktas-Kutay-Mendlovic 1996 or Candan 2000 algorithm.
- ☑ Verify `cargo test --workspace` 0 failures; `cargo clippy --workspace --all-targets ---D warnings` 0 warnings.

Performance & Native GPU Precision phase

- ☑ Add `NufftWgpuBackend::execute_fast_type1_1d_with_buffers`, `execute_fast_type2_1d_with_buffers`, `execute_fast_type1_3d_with_buffers`, `execute_fast_type2_3d_with_buffers` façade methods to `apollo-nufft-wgpu/src/infrastructure/device.rs`.
- ☑ Add Criterion bench target `buffer_reuse` and `benches/buffer_reuse.rs` to `apollo-nufft-wgpu`; covers fast Type-1/Type-2 1D per-call vs with-buffers across N=64,128,256.
- ☑ Add Criterion bench target `buffer_reuse` and `benches/buffer_reuse.rs` to `apollo-fft-wgpu`; covers 3D FFT forward/inverse per-call vs with-buffers across nx=ny=nz=4,8,16.
- ☑ Add `native-f16` feature flag to `apollo-fft-wgpu/Cargo.toml`.
- ☑ Create `src/infrastructure/shaders/fft_native_f16.wgsl` and `pack_native_f16.wgsl` with `enable f16;`, `array<f16>` buffers, f16 twiddle factors, and f16 butterfly passes.
- ☑ Create `src/infrastructure/gpu_fft/f16_plan.rs` with `GpuFft3dF16Native`, `try_new`, `try_from_device`, `forward_native_f16`, `inverse_native_f16`, `device_supports_f16`, and `validate_dimensions_f16`.
- ☑ Expose `GpuFft3dF16Native` from `src/infrastructure/gpu_fft/mod.rs` and `src/lib.rs` under `#[cfg(feature = "native-f16")]`.
- ☑ Add `native_f16_forward_matches_f32_within_f16_tolerance_when_device_exists` test: $|\text{error}| < 5 \times 10^{-3}$ bound derived from $O(\log N) \cdot \varepsilon_{f16}$ with N=4.
- ☑ Document radix-2-only constraint for `GpuFft3dF16Native` (Bluestein chirp f16 shader deferred); ADR: twiddles computed in f32 then narrowed to f16 to bound two-source accumulation error.
- ☑ Verify `cargo check --workspace --all-targets` clean (default features).
- ☑ Verify `cargo check --package apollo-fft-wgpu --all-targets --features native-f16` clean.
- ☑ Verify `cargo test --workspace --all-targets` passes 465 tests, 0 failures.
- ☑ Verify `cargo clippy --workspace --all-targets` zero errors and zero warnings.
- ☑ Verify `cargo test --package apollo-fft-wgpu --features native-f16` passes 9 tests including new native-f16 parity test.

Closure II phase (fixture expansion, capability table, documentation sync)

- ☑ Add `ntt_n8_impulse_fixture` to `apollo-validation` published-reference suite: NTT8([1,0,0,0,0,0,0,0])=[1,1,1,1,1,1,1,1] (Pollard 1971 impulse theorem, N=8 generalization); verified at PUBLISHED_FIXTURE_LIMIT=1×10^{-12}.
- ☑ Add `ntt_polynomial_convolution_fixture` to `apollo-validation` published-reference suite: INTT(NTT([1,2,0,0])⊙NTT([3,4,0,0]))=[3,10,8,0] from $(1+2x)(3+4x)=3+10x+8x^2$ (Pollard 1971 Convolution Theorem); pointwise product computed via 128-bit widening mod 998244353; verified at PUBLISHED_FIXTURE_LIMIT=1×10^{-12}.
- ☑ Add `nufft_quarter_period_phase_fixture` to `apollo-validation` published-reference suite: Type-1, single unit source at x=L/4, N=4 → F=[1,-i,-1,i]; derived from exp(-πi·k_signed/2) with k_signed ∈ {0,1,2,-1}; max f64 trig error < $2\times10^{-16} \ll 1\times10^{-12}$ threshold (Dutt and Rokhlin 1993).
- ☑ Update `run_published_reference_suite` vec to include `ntt_n8_impulse_fixture`, `ntt_polynomial_convolution_fixture`, and `nufft_quarter_period_phase_fixture` (total 10 fixtures).
- ☑ Update fixture-count assertions from 7 to 10 in `validation_suite_produces_value_semantic_reports` and `published_reference_suite_checks_computed_fixture_values`.
- ☑ Add `use apollo_ntt::{intt, NttPlan, DEFAULT_MODULUS};` to `apollo-validation/src/application/suite.rs` imports.
- ☑ Add Mixed-Precision Capability Table section to `ARCHITECTURE.md` covering all 35 crates with advertised profile, supported storage, GPU compute precision, and notes; includes native-f16 and NTT precision contract subsections.
- ☑ Update `README.md`: document `native-f16` feature completion (radix-2 and Bluestein, `GpuFft3dF16Native`, `O(log N)·ε_f16` bound), updated WGPU mixed-precision surface, and 10-fixture validation suite reference.
- ☑ Verify `cargo test --package apollo-validation --lib -- tests` passes 3 tests with `attempted = 10`.
- ☑ Verify `cargo test --workspace --all-targets` zero failures after Closure II changes.
- ☑ Verify `cargo clippy --workspace --all-targets -- -D warnings` zero errors and zero warnings after Closure II changes.

Gap-Closure & Extension phase (Bluestein f16, 3D NUFFT bench, published fixtures)

- ☑ Create `chirp_native_f16.wgsl` with `enable f16;`, `array<f16>` for all four storage bindings (data_re, data_im, chirp_re, chirp_im), f32-precision twiddle narrowed to f16, and no-op `chirp_scale` matching the f32 contract.
- ☑ Remove power-of-two-only restriction from `validate_dimensions_f16`: keep N ≥ 2 requirement only.
- ☑ Add `f16_next_pow2`, `f16_axis_strategy`, `f16_axis_workspace_elems` free functions to `f16_plan.rs`.
- ☑ Add `use crate::infrastructure::gpu_fft::strategy::{AxisStrategy, ChirpData};` to `f16_plan.rs` imports.

- ☑ Add `strategy_x/y/z: AxisStrategy` and `chirp_x/y/z: Option<ChirpData>` fields to `GpuFft3dF16Native`.
- ☑ Update `try_from_device` workspace buffer sizing to `max(f16_axis_workspace_elems per axis) × 2 bytes` to accommodate Bluestein padded lengths.
- ☑ Update `build_axis_pack` calls in `try_from_device` to pass `m` as `fft_len` for ChirpZ axes and `n` for Radix2 axes.
- ☑ Update `RadixStages` construction in `try_from_device`: `RadixStages::empty()` for ChirpZ axes, `precompute` for Radix2 axes.
- ☑ Add Bluestein chirp data construction for each axis in `try_from_device` using `build_chirp_data_f16`.
- ☑ Add `strategy_x/y/z` and `chirp_x/y/z` to the `Ok(Self { … })` return in `try_from_device`.
- ☑ Add `build_chirp_data_f16` private method: computes h in f32, narrows to f16 u16 bits, creates f16 chirp buffers, builds `data_chirp_layout`/`data_chirp_bg`, compiles `chirp_native_f16.wgsl` pipelines, returns `ChirpData` with embedded `radix2_fwd`/`radix2_inv`.
- ☑ Add `dispatch_chirp_f16` private method using flat 1D dispatch `(total).div_ceil(256), 1, 1` throughout, eliminating the data-race risk present in the original f32 `dispatch_chirp` (which uses 2D workgroup dispatch with a flat-index shader).
- ☑ Update `run_f16_axis_fft` to match-dispatch on `strategy_x/y/z`: `dispatch_radix2` for Radix2, `dispatch_chirp_f16` for ChirpZ.
- ☑ Add `non_pow2_f16_forward_inverse_roundtrip_when_device_exists` test: 3×3×3 Bluestein path, roundtrip error < 0.05 (analytically bounded by $O(\log_2 4)\cdot\varepsilon_f16\cdot 2\cdot 3 \approx 1.2\times 10^{-2}$).
- ☑ Add `bench_fast_type1_3d` and `bench_fast_type2_3d` Criterion functions to `apollo-nufft-wgpu/benches/buffer_reuse.rs`; covers per-call vs `with_buffers` for 3D fast NUFFT across N=4,6,8 using `NufftGpuBuffers3D` and `NufftWgpuPlan3D`.
- ☑ Fix approximate-TAU clippy warnings in `apollo-nufft-wgpu/benches/buffer_reuse.rs`: replace `6.283` literals with `std::f32::consts::TAU` and `std::f32::consts::PI`.
- ☑ Add `use apollo_ntt::ntt;` import and `apollo-ntt` path dep to `apollo-validation`.
- ☑ Add `ntt_impulse_fixture` (NTT([1,0,0,0])→[1,1,1,1], Pollard 1971 impulse theorem) to `apollo-validation` published-reference suite.
- ☑ Add `ntt_constant_fixture` (NTT([1,1,1,1])→[4,0,0,0], DFT-of-constant geometric-series theorem) to `apollo-validation` published-reference suite.
- ☑ Add `nufft_impulse_at_origin_fixture` (Type-1 single source x=0, value=1 → F[k]=1 ∀k, Dutt and Rokhlin 1993) to `apollo-validation` published-reference suite.
- ☑ Update `run_published_reference_suite` to include the three new fixtures (total 7).
- ☑ Update fixture-count assertions from 4 to 7 in `validation_suite_produces_value_semantic_reports` and `published_reference_suite_checks_computed_fixture_values`.
- ☑ Verify `cargo check --workspace --all-targets` clean (default features).
- ☑ Verify `cargo check --package apollo-fft-wgpu --features native-f16 --all-targets` clean.
- ☑ Verify `cargo clippy --workspace --all-targets` zero errors, zero warnings.
- ☑ Verify `cargo test --workspace --all-targets` zero failures.
- ☑ Verify `cargo test --package apollo-fft-wgpu --features native-f16` passes 10 tests including `non_pow2_f16_forward_inverse_roundtrip_when_device_exists`.

Closure phase

- ☑ Fix `[workspace.lints.clippy]` priority: assign `all`/`pedantic` groups `priority = -1` so individual overrides take precedence.
- ☑ Propagate workspace lints to all 39 crates via `[lints] workspace = true`.
- ☑ Add comprehensive DSP-appropriate pedantic suppressions to workspace lints.
- ☑ Fix `apollo-fft` doc-lint and needless_range_loop warnings in `direct.rs`.
- ☑ Replace `CpuBackend::default()` with `CpuBackend` in transport tests.
- ☑ Add `#![allow(missing_docs)]` and doc comments to benchmark file.
- ☑ Add `fast_type2_1d_normalization_invariance_when_device_exists` test.
- ☑ Add normalization convention docs to WGSL shaders and `encode_inverse_split`.
- ☑ Remove 22 scratch/temporary files from repository root and `scratch/` directory.
- ☑ Add scratch-file gitignore patterns.
- ☑ Verify zero clippy errors, zero clippy warnings, zero test failures.
- ☑ Read workspace metadata, README, crate manifests, and validation gaps.
- ☑ Classify current rename state as authoritative without reverting user changes.
- ☑ Add all Apollo crates to workspace membership.
- ☑ Fix compile blockers across validation, Python bindings, and missing transform crate roots.
- ☑ Replace incomplete validation suite with real computed report paths.
- ☑ Fix CZT, SFT, and STFT defects found by bounded tests.
- ☑ Move SFT domain model, plan execution, direct kernel, and tests into the authoritative `apollo-sft` crate hierarchy.
- ☑ Verify `apollo-fft/src` has no SFT implementation or SFT export path.
- ☑ Split validation dependencies so optional `rustfft` is enabled only through `apollo-validation/external-references`; audited that `realfft` is absent from the workspace dependency graph.
- ☑ Complete the new multi-crate validation API for `apollo-validation`.
- ☑ Fix `FftPlan1D`/`FftPlan2D` missing `forward_complex`/`inverse_complex` wrappers.
- ☑ Implement `kernel::radix2` (iterative Cooley-Tukey DIT, power-of-2) with value-semantic tests.
- ☑ Implement `kernel::bluestein` (chirp-Z, arbitrary N, verified for N=3,5,6,7,11) with value-semantic tests.

- ☑ Add `fft_forward_64`, `fft_inverse_64`, `fft_inverse_unnorm_64`, `fft_forward_32`, `fft_inverse_32`, `fft_inverse_unnorm_32` auto-selecting wrappers to `kernel::mod`.
- ☑ Update `FftPlan1D`, `FftPlan2D`, `FftPlan3D` axis-pass methods to use new O(N log N) kernel.
- ☑ Run `cargo test --workspace` and verify zero failures.
- ☑ Add `apollo-hilbert` with Hilbert transform plans, analytic signal, envelope/phase APIs, docs, and tests.
- ☑ Add `apollo-radon` with parallel-beam Radon plans, sinogram storage, backprojection, filtered backprojection, docs, and tests.
- ☑ Complete `apollo-mellin` execution APIs and analytical tests.
- ☑ Replace stale skeleton crate documentation for completed transform crates.
- ☑ Add DCT/DST direct-kernel value-semantic tests.
- ☑ Remove incorrect DCT/DST fast branch and keep large-plan direct parity tests.
- ☑ Add Python `rfft3`/`irfft3` value-semantic tests.
- ☑ Add validation report JSON schema-shape tests.
- ☑ Add Criterion benchmarks for FFT kernel strategy.
- ☑ Add caller-owned Radon ramp-filter path and parity test.
- ☑ Update FFT 1D/2D/3D Rustdoc and README ownership text to match radix-2/Bluestein execution.
- ☑ Remove duplicate transformed-lane collections from FFT 2D/3D axis passes.
- ☑ Replace NUFFT 3D per-lane allocation and NUFFT 1D type-2 grid copying with reusable/borrowed buffers.
- ☑ Add CZT README, Bluestein proof sketch, forward_into parity test, and remove CZT product-vector copy.
- ☑ Add FWHT README, Hadamard theorem/proof sketch, real/complex `*_into` APIs, and caller-owned parity tests.
- ☑ Add NTT README, root-of-unity theorem/proof sketch, true in-place paths, `*_into` APIs, residue-normalization tests, and overflow-safe modular addition.
- ☑ Add FrFT README, rotation theorem/proof sketch, finite integer-order state, inverse APIs, and inverse/caller-owned parity tests.
- ☑ Add STFT README, overlap-add theorem/proof sketch, clean filler comments, replace oversized expect text, and add inverse_into parity coverage.

- ☑ Add DCT/DST README, inverse-pair theorem/proof sketch, inverse_into API, and caller-owned inverse parity tests.
- ☑ Repair SFT non-UTF-8 Rustdoc byte, replace deprecated ndarray extraction, and route SFT direct-reference tests through the owning kernel.
- ☑ Restore `NttPlan` after truncation and verify NTT value/property tests.
- ☑ Move CZT tests out of the plan impl, add `num-complex` serde support, and reject zero `W`.
- ☑ Repair SHT invalid UTF-8 reference markers.
- ☑ Fix SDFT `Result` propagation and QFT property-test dimension construction.
- ☑ Remove duplicated NUFFT 3D module content and restore type-2 sorted-position interpolation.
- ☑ Replace NUFFT Kaiser-Bessel `I_0` polynomial approximation with the defining convergent series.
- ☑ Replace Wavelet Morlet approximate-admissibility note with a DC-corrected kernel and zero-mean test.
- ☑ Ensure each `crates/apollo-*` crate has a crate-local README with architecture, mathematical contract, and verification notes.
- ☑ Rename dense FFT WGPU crate to `apollo-fft-wgpu` and update validation/Python dependencies.
- ☑ Add `apollo-nufft-wgpu` with capability, plan, and unsupported-execution contracts.
- ☑ Add WGPU backend-boundary crates for all remaining transform domains.
- ☑ Verify each new WGPU crate has domain, application, infrastructure, verification, and README artifacts.
- ☑ Run `cargo fmt --all -- --check`, `cargo check --workspace --all-targets`, and `cargo test --workspace --all-targets`.
- ☑ Eliminate per-stage `Vec<Complex>` twiddle allocations in `radix2` forward/inverse f32/f64 by replacing with a single N/2-entry precomputed stride-indexed table.
- ☑ Cache Bluestein scratch buffer in `FftPlan1D` via `Mutex<Vec<Complex64>>` to eliminate per-call allocation on the Bluestein hot path.
- ☑ Precompute DWT highpass QMF coefficients once per `analysis_stage_into`/`synthesis_stage_into` call using the Smith-Barnwell QMF identity.
- ☑ Add Parseval/Plancherel energy-invariance theorem (with proof sketch) to `radix2.rs` module doc and Unified Twiddle Table theorem.
- ☑ Add I_0 convergence theorem and K=256 sufficiency corollary to `kaiser_bessel.rs`.
- ☑ Derive and verify a correct FFT-based DCT/DST acceleration strategy.

- ☑ Add published-reference validation fixtures for DFT, DHT, DCT-II, and DST-II in `apollo-validation`.
- ☑ Add WGPU NUFFT direct Type-1/Type-2 1D/3D numerical kernels and parity tests.
- ☑ Add direct forward CZT WGPU kernels with CPU parity validation.
- ☑ Add forward Hilbert WGPU kernels with CPU parity validation.
- ☑ Add forward Mellin WGPU kernels with CPU parity validation.
- ☑ Add forward and inverse NTT WGPU kernels with CPU parity validation.
- ☑ Add forward and inverse GFT WGPU kernels with CPU parity validation.
- ☑ Add forward and inverse QFT WGPU kernels with CPU parity validation.
- ☑ Add forward Radon WGPU kernels with CPU parity validation.
- ☑ Add numerical DCT-II/DCT-III/DST-II/DST-III WGPU kernels with CPU parity validation.
- ☑ Add numerical DHT WGPU kernels with CPU parity validation.
- ☑ Add numerical FWHT WGPU kernels with CPU parity validation.
- ☑ Add numerical WGPU kernels to transform-specific WGPU crates with CPU parity validation (QFT, FrFT, SDFT, GFT, STFT, Wavelet DWT, SFT, and SHT implemented).
- ☑ Add forward and inverse unitary QFT WGPU kernels with CPU parity validation (tol 1e-3).
- ☑ Add forward and inverse chirp-kernel FrFT WGPU kernels with 5-mode dispatch and CPU parity validation.
- ☑ Add forward direct-bins SDFT WGPU kernels with CPU parity validation against SdftPlan::direct_bins.
- ☑ Add forward and inverse GFT WGPU dense-matmul kernels with caller-supplied basis and CPU parity validation.
- ☑ Add forward Hann-windowed STFT WGPU kernels with CPU parity validation.
- ☑ Add forward and inverse Haar DWT WGPU kernels with roundtrip and Parseval energy validation.
- ☑ Add SFT WGPU direct dense DFT forward/inverse execution with sparse top-K CPU parity validation.
- ☑ Add SHT WGPU direct spherical harmonic execution without duplicating CPU-owner basis/quadrature logic.
- ☑ Move SHT WGPU associated Legendre recurrence, harmonic normalization, conjugation, and quadrature weighting into a GPU basis-generation pass.
- ☑ Add NUFFT WGPU fast 1D gridding execution after direct 1D/3D coverage.

- ☑ Add NUFFT WGPU fast 3D gridding execution after fast 1D parity coverage.
- ☑ Audit `realfft` references and document that no additional feature gate is required because `realfft` is not a workspace dependency.
- ☑ Audit remaining transform crates against published references and cross-crate validation fixtures.
- ☑ Fix hardcoded `type2_1d_max_relative_error = 0.0` mock in apollo-validation by computing actual fast vs exact NUFFT type-2 relative error.
- ☑ Add CZT independent DFT cross-check test (CZT vs apollo_fft on same input, not just CZT vs direct-CZT).
- ☑ Add NUFFT uniform-grid DFT equivalence test (type-1 at x_j=j*L/N equals DFT(c)).
- ☑ Replace existence-only Morlet CWT test with value-semantic resonance test.
- ☑ Add DHT–DFT relationship cross-check (H[k] = Re(F[k]) - Im(F[k])).
- ☑ Remove host-side zero upload for `apollo-sht-wgpu` generated basis storage.
- ☑ Fix GPU fast type-2 1D NUFFT normalization: pack deconv values scaled by `oversampled_len` in `execute_fast_type2_1d` to compensate for `encode_inverse_split` normalized IFFT (÷m), matching the CPU `type2_into` ×m rescaling without an extra host vector.
- ☑ Remove host-side zero uploads for inactive `apollo-nufft-wgpu` fast-path bind-group placeholders.
- ☑ Remove full-field lane-copy allocation from contiguous 2D row and 3D innermost FFT axis passes.
- ☑ Add value-semantic coverage for caller-owned 3D typed FFT execution across `f64`, `f32`, and mixed `f16` profiles.
- ☑ Add validation benchmark timing coverage for forward and inverse `f64`, `f32`, and mixed `f16` FFT profiles.
- ☑ Add value-semantic DHT and DCT/DST typed storage coverage for `f64`, `f32`, mixed `f16`, and profile mismatch rejection.
- ☑ Add value-semantic FWHT typed storage coverage for `f64`, `f32`, mixed `f16`, and profile mismatch rejection.
- ☑ Verify all 39 workspace crates have manifests, READMEs, and library roots; repair the missing `apollo-python` architecture and verification README sections.
- ☑ Add mixed-precision CPU storage contracts to remaining eligible transform crates: NUFFT and SHT.
- ☑ Add mixed-precision capability contracts or explicit unsupported records to WGPU crates: FFT-WGPU, CZT-WGPU, DCTDST-WGPU, DHT-WGPU, FrFT-WGPU, FWHT-WGPU, GFT-WGPU, Hilbert-

WGPU, Mellin-WGPU, NTT-WGPU, NUFFT-WGPU, QFT-WGPU, Radon-WGPU, SDFT-WGPU, SFT-WGPU, SHT-WGPU, STFT-WGPU, and Wavelet-WGPU.

- ☑ Add `NufftGpuBuffers1D`/`NufftGpuBuffers3D` reusable GPU buffer structs and `execute_fast_*_with_buffers` methods.
- ☑ Add `NufftPlan3D::type2_into` zero-allocation 3D Type-2 path.
- ☑ Add value-semantic typed verification tests for `apollo-nufft` covering Complex64, Complex32, [f16;2], and profile mismatch.
- ☑ Add `apollo-fft-wgpu` reusable GPU buffer structs for repeated 3D FFT dispatch.
- ☑ Add `apollo-ntt-wgpu` reusable GPU buffer structs for repeated direct NTT dispatch.
- ☑ Add `apollo-ntt-wgpu` reusable-buffer quantized `u32` forward/inverse dispatch.
- ☑ Add debug-gated GPU grid readbacks (after load, after IFFT) behind a `cfg(test)` feature in `apollo-nufft-wgpu` for faster future numerical triage.
- ☑ Set up CI pipeline for lint and test regression prevention.
- ☑ Add real mixed-precision WGPU execution where existing `f32` shader kernels support typed host-storage promotion/quantization: FFT-WGPU 3D, NUFFT-WGPU direct/fast 1D/3D, DHT-WGPU, FWHT-WGPU, CZT-WGPU, DCTDST-WGPU, FrFT-WGPU, GFT-WGPU, Hilbert-WGPU, Mellin-WGPU, QFT-WGPU, Radon-WGPU, SDFT-WGPU, SFT-WGPU, SHT-WGPU, STFT-WGPU, and Wavelet-WGPU now expose verified typed storage paths.
- ☑ Remove inactive cudatile crate and Python backend report surface.
- ☑ Keep NTT-WGPU floating mixed precision explicit-unsupported and add exact quantized `u32` residue storage instead.
- ☑ Reuse `NttGpuBuffers` for exact quantized `u32` NTT-WGPU dispatch to eliminate repeated buffer/bind-group/staging allocation.
- ☑ Add typed CZT caller-owned storage coverage for `Complex64`, `Complex32`, mixed `[f16; 2]`, and profile mismatch rejection.
- ☑ Add typed FrFT caller-owned storage coverage for `Complex64`, `Complex32`, mixed `[f16; 2]`, and profile mismatch rejection.
- ☑ Add typed GFT caller-owned storage coverage for `f64`, `f32`, mixed `f16`, inverse roundtrip, and profile mismatch rejection.
- ☑ Add typed Hilbert caller-owned quadrature coverage for `f64`, `f32`, mixed `f16`, analytic real-part preservation, and profile mismatch rejection.
- ☑ Add typed Mellin caller-owned resample coverage for `f64`, `f32`, mixed `f16`, represented-input moments/spectra, and profile mismatch rejection.

- ☑ Add typed QFT caller-owned storage coverage for `Complex64`, `Complex32`, mixed `[f16; 2]`, inverse roundtrip, and profile mismatch rejection.
- ☑ Add typed Radon caller-owned forward/backprojection coverage for `f64`, `f32`, mixed `f16`, represented-input projection parity, and profile mismatch rejection.
- ☑ Add typed SDFT caller-owned direct-bin coverage for `f64`/`Complex64`, `f32`/`Complex32`, mixed `f16`/`[f16; 2]`, represented-input parity, and profile mismatch rejection.
- ☑ Add typed STFT caller-owned forward/inverse coverage for `f64`/`Complex64`, `f32`/`Complex32`, mixed `f16`/`[f16; 2]`, represented-input parity, and profile mismatch rejection.
- ☑ Add typed Wavelet DWT/CWT caller-owned coverage for `f64`, `f32`, mixed `f16`, represented-input parity, DWT inverse roundtrip, and profile mismatch rejection.
- ☑ Add typed SFT sparse forward/inverse caller-owned coverage for `Complex64`, `Complex32`, mixed `[f16; 2]`, represented-input parity, inverse roundtrip, sparse shape rejection, and profile mismatch rejection.
- ☑ Add typed SHT real/complex caller-owned coverage for `f64`/`Complex64`, `f32`/`Complex32`, mixed `f16`/`[f16; 2]`, represented-input parity, inverse roundtrip, shape rejection, and profile mismatch rejection.
- ☑ Add typed NUFFT 1D/3D Type-1/Type-2 caller-owned coverage for `Complex64`, `Complex32`, mixed `[f16; 2]`, represented-input parity, Type-2 parity, shape rejection, and profile mismatch rejection.